

Automating the Verification of Software Testability of Software Artifacts

Virgil Ribeyre

Laboratoire d'Informatique de Grenoble (LIG-UGA)

Grenoble, France

virgil.ribeyre@protonmail.com

Supervised by: Lydie du Bousquet

I understand what plagiarism entails, and I declare that this report is my own, original work.

Name, date and signature: Virgil Ribeyre, 5th June 2023.

Abstract

In software programming, roughly 40% of the total time and cost is spent on *testing*, that is, executing a program with the intent of finding errors. Depending on the quality, conception and domain of the existing application, it may be more testable than others. We define that as *testability*: the relative ease and expense to revealing software faults. In the literature, testability is measured using either metrics or by the presence of code antipatterns, or *code smells* indicating a deeper problem in a software artifact. Some of these code smells often impact testability negatively, increasing costs on testing, usually pushing developers away from it. By interviewing professionals of the industry and through a literature review, we devised a list of 27 rules to use in a static code analysis tool. This tool, a *linter* prototype, can point out 3 of these code smells on java code bases, thus preventing these antipatterns to take root even before the testing phase of the software begins.

1 Introduction

In software engineering, testing is executing a program with the intent of finding errors. It is the most used validation technique in the industry. In a typical programming project, it is estimated that more than 40 percent of total costs and time are spent on software testing [Myers *et al.*, 2011].

Unfortunately, as much as this step is crucial to ensure software quality and maintainability, it is usually considered a "dark art" of software engineering [Myers *et al.*, 2011].

As such, being able to identify practices and **testable** systems is crucial to ensure quality during the entire system life-cycle. Binder defines *testability* as the relative ease and expense to revealing software faults [Binder, 1984]. Ensuring testability requires knowledge, rigorous development processes (such as Test Driven Design) and thorough verifications by software integrators and maintainers. More than often verifications are overlooked in favor of implementing re-

quired features as soon as possible. However, some practices might be enforced automatically using static code analysis.

Traditionally, in literature, testability is articulated around two dimensions: metrics and patterns. Metrics are used to give a quantitative indication on testability and assess the difficulty in making tests for a software artifact. Multiple metrics were proposed to measure testability, for instance the *number of line of code*, *number of operators* or *cyclomatic complexity* [Shaheen, 2009]. Unfortunately these metrics do not give a granular evaluation of testability, rather a global value, limiting its usage in a day-to-day software development process.

Another approach is avoiding set of antipatterns that hinders testability without necessarily being a bug [Shaheen, 2009]. This empirical approach is much more suitable to software development processes, but its reach is more general than just testability in a software artifact.

In the software development industry, we use **linters**. A linter helps enforce and validate the usage of coding practices and patterns to simplify integration and maintenance while ensuring scalability. It is usually a static code analysis tool validating source code by checking a set of selected rules each time the code base is built or ran. ESLint [ESLint, 2023a] and SonarQube [SonarQube, 2023b] are the most used linters to date [Tómasdóttir *et al.*, 2020] [SonarQube, 2023c].

The *goals* of this paper are to define a set of rules and antipatterns we can statically check for in a code base to ensure a high level of testability and developing our own linter to enforce these rules. More specifically, this study aims at addressing the two following questions:

- **RQ₁**: *Is it possible to enforce testability oriented rules using a linter?* This question aims at investigating to what extent we are able to check statically for patterns improving testability in a code base and enforce them using our own set of rules.
- **RQ₂**: *Which patterns should we check for in a software artifact to further increase its testability?* This question depends directly on a positive response to **RQ₁** and follows its continuity in identifying statically verifiable patterns.

In the following, section 2 presents our research methodology to collect rules Section 3 compares the existing linters used in the industry: ESLint [ESLint, 2023a] and SonarQube

[SonarQube, 2023b]. Section 4 gathers and selects the patterns enforceable with our custom linter. Section 5 presents the current design of the linter and its features. Section 6 presents and discusses the results obtained and section 7 concludes this study and gives perspectives.

2 Collecting Rules

To answer **RQ₁**, it is important to gather testability antipatterns and patterns, which means finding coding structures having a direct or indirect impact on testability. Binder defines one of the factor of testability as the implementation of the software artifact, that is how one writes its source code [Binder, 1984]. In the case of automation, it is possible to find a set of implementations, or **patterns**, increasing testability in a software artifact. Using these patterns, we will try expressing them as rules a linter is able to verify on a code base [du Bousquet, 2010].

It can then be integrated in the testing processes as a built-in test tool to point out potential software flaws hindering testability. However, we need to be exhaustive in our rules set to make it both widely usable, yet precise enough to find a lack of testability, reliably. As such, we need to gather these practices then formalize them into usable rules.

We used two strategies to collect patterns we describe thereafter: in section 2.1, we individually interview professionals in the software development industry using an interview template we made. In section 2.2 we analyze the limits of our interview methodology and explore possible enhancements to it. Finally, in section 2.3 we review literature on code antipatterns and testability patterns.

2.1 Interviews

Gathering these antipatterns and patterns increasing testability of a software artifact can be done by interviewing software engineers or developers. Thus, we can better understand their approaches on software testing as well as their observations on what makes a software artifact testable.

Part of my work in this internship consisted in building an interview template. This template contains four sections. *Interviewé* describes the personal information of the interviewee as well as their experiences, and the path to the recording of the interview if consented. *Check-list préliminaire* describes a checklist I used before starting the interview ensuring I describe the study conducted, enumerate the data collected, ask for consent in the recording and ask for consent to cite the company they are currently working with. *Description de l'étude* contains a brief explanation of the study conducted for the interviewee. *Questions* contains the questions for the interviewee and their replies. The difficulty was to make questions open enough to inhibit their tendencies to censor themselves while still retaining clear boundaries to make the results usable. The interview template is available in the Appendix of this report (appendix A.2).

The interview is in French because its first version was targeted to ex-colleagues or clients I used to work with for simplicity when sourcing my network. This interview was validated by my supervisor Lydie du Bousquet.

During these interviews it is also crucial to gather what languages and frameworks the interviewees use, to target which

languages should we check for in the linter. In total, I interviewed 2 senior software engineers, 6 junior developers and 2 mid-developers.

Interviews lasted from an hour to an hour and a half. Juniors were unable to provide a single testability pattern or antipatterns when asked and generally did not test as much in their day-to-day work, having an outlook on testing being similar to "dark magic" and expressing a form of bewilderment when asking for level of tests done in their company. Mid and Seniors were able to provide one or two antipatterns in average. These antipatterns usually boiled down to a lack of code legibility or describing functions, methods or classes breaking the Single Responsibility Principle. Specific patterns hindering testability were not reported clearly besides two concrete examples which were given by seniors:

- **Clever Code:** a code working as intended but being generally hard to understand at a glance [Jerzyk and Madeyski, 2023].
- **Using Meta-programming:** a single instruction generating deep logic at compile time. A prime example is the usage of function templates in C++ [cplusplus.com, 2023].

Interviews provide a simple way to gather a smaller set of practices from professional experiences, but they are not exhaustive enough to further define a list of practices to enforce statically. This is why we reviewed the current state of the literature on the subject of testability and *patterns* as well as the state-of-the-art on linters.

2.2 Limits and Possible Extensions to the Interviews

The interviews are a valuable source of information. However, they give a limited reflection time to allow the interviewee to explain each and every antipatterns or patterns they might have found in their experiences. A possible solution would be to define an online survey that people can fill in every time they see or think of such patterns.

We did not go that route due to a lack of time in the internship and also a lack of relevancy: the major problem of these interviews was the lack of understanding of testability as a whole. If we were able to conduct that endeavor, we might face the same problems as with the interview. Also, online surveys are generally quickly filled and could lead to a lack of precision rendering the results more difficult to analyze.

2.3 Patterns in literature

Our main focus for practices gathering was to find recurring antipatterns in software artifacts to statically check for. In light of these patterns, Refactoring, Second Edition, by Martin Fowler, is a reference as it describes and names multiple *code smells* [Fowler, 2018].

In his book, Fowler defines a code smell as "a surface indication that usually corresponds to a deeper problem in the system" [Fowler, 2018]. A code smell is not a bug in a program that prevents it to run, but it is an architectural problem, spanning from a single block in the software artifact to a greater extent, contributing to a reduced maintainability and a more error-prone program.

Following in that idea, we studied and classified code smells that could reveal a lack of testability in a software artifact. These smells were gathered from different sources. The method used to classify them into a usable set of rule is explained in section 4.

3 Existing Linters

To further our review of the existing, we dressed up a comparison between SonarQube [SonarQube, 2023b] and ESLint [ESLint, 2023a] to avoid falling in the pitfall of making yet another general purpose static code checker.

While SonarQube [SonarQube, 2023b] and ESLint [ESLint, 2023a] are both linters, they do not provide the same features and SonarQube uses have another whole extent as it is also a continuous integration checker [SonarQube, 2023b]. As such, Table 1 compares these linters using the following list of normalized criterion. We did not take in account simplicity when it comes to creating, managing and modifying a configuration for both SonarQube [SonarQube, 2023b] and ESLint [ESLint, 2023a] as they differ too much to define a normalized criterion for it.

- The languages covered by the linter
- Does it provide formatting rules? (e.g. ensuring indent has the required size and style)
- Is it able to detect code smells?
- Is it able to detect vulnerability? (e.g. Buffer overflows in C programs, API keys being in clear in the code, etc.)
- Is it able to perform automatic code formatting compliant to its rules?
- Is it able to auto-fix reported errors?
- How many rules does it provide? We did not take in account the removed or deprecated rules for that number.
- What kinds of rule error levels does it provide?
- Are its capabilities expendable (e.g. via plugins, custom rules, custom formatters or custom parsers, etc.)
- Is it able to compute test coverage of a software artifact?
- Can we integrate it through Continuous Integration (CI) pipelines?
- Does it analyze Pull Requests?
- Does it provide the possibility to analyze branches from a version control system?

When making our own linter, we will stick to a simpler approach when it comes to configuration, following the principles established by ESLint [ESLint, 2023a] minus its extensibility to reduce the scope of the linter for its first release.

4 Code smells hindering testability

In this section, we define and analyze a list of code smells hindering directly testability. Using these code smells we define a set of rules to be implemented in our linter, thus answering **RQ₂**.

Defining rules for the linter requires finding and classifying code smells directly, or indirectly, hindering testability.

Fortunately, Marcel Jerzyk presents a classification and taxonomy of 56 code smells in his study and website¹ based on Fowler’s [Fowler, 2018] and Wake’s [Wake, 2004] works [Jerzyk and Madeyski, 2023]. We add to that list two code smells reported in du Bousquet’s article:

- **Always Have Setters and Getters:** Setters and Getters are important when testing a class in object-oriented programming and enforce automatic checks when accessing or modifying attributes. They should always be present, at least for test purposes [du Bousquet, 2010].
- **No Void Method/Function:** `void` methods and functions are part of a programming language as a whole but shadow useful information, for instance, if the method accessing a hash map found an item using the provided key. As such, for test purposes, this rule disallows the use of `void` as a return type in methods and functions [du Bousquet, 2010].

This give us a list of 58 code smells to select from.

To create our list of rules for the linter, we will proceed in two steps:

1. **Selecting smells hindering testability.** This selection requires a granular analysis of each smell to be sure it hinders testability based on our interviews and literature review. For example, the smell *Inconsistent Style* does not hinder testability as much as it hinders code readability. As such, we will not add a rule to check for it.
2. **Selecting smells allowing static analysis.** Using the previous selection, we will select empirically smells which can be statically analyzed with a linter. That is, we can systematically check for them in a code base. For example, the smell *Binary Operator in Name* falls in that principle since it requires checking for the name of a method or function to not have `or` or `and` in their names. The linter will be able to easily find this smell by checking the name. As such `DoThisAndThat()` should not be compliant and be flagged by the linter.

The first step narrowed the list to 49 smells directly hindering testability. This list was narrowed again during the second step to define a final list of 27 rules implementable for our linter. You can find the list of rules in subsection A.1 in the Appendix of this report.

The most common problem when testing is having to write tests for a part of the software having multiple responsibilities at once. We define such codes as violators of the Single Responsibility Principle (SRP). *Responsibility Smells*, *Measured Smells* and *Conditional Logic Smells* fall in that violation [Jerzyk and Madeyski, 2023]. As such, they increase complexity when testing as it requires testing each and every responsibility of a single class, method or function generally reducing how exhaustive a test must be. Multiple responsibilities allocated to a single function, method or class create hard to test environments [Fowler, 2018] [Myers *et al.*, 2011]. An example of that behavior could be *Conditional Complexity* where a function or method addresses multiple cases using a `switch` or `if-else` construction [Fowler, 2018]. Indeed,

¹<https://luzkan.github.io/smells/>

Criterion	SonarQube	ESLint
Languages Covered	ABAP, Apex, C, C++, CloudFormation, Cobol, C#, CSS, Docker, Flex, Go, HTML, Java, JavaScript (Via ESLint [SonarQube, 2023b].), Kotlin, Kubernetes, Objective-C, PHP, PL/I, PL/SQL, Python, RPG, Ruby, Scala, Swift, Terraform, Typescript (Via ESLint [SonarQube, 2023b].), T-SQL, Visual Basic .NET, Visual Basic 6, XML	JavaScript, Typescript, Extended JavaScript Syntax (Via plugin [ESLint, 2023a]) (e.g. Flowtypes)
Code formatting rules	Yes	Yes
Code smells detection	Yes	Yes
Vulnerability detection	Yes	Yes
Formatting	Yes	Yes
Auto-fix reported errors	Yes (for some errors)	Yes (for some errors)
Number of Rules	5139 [SonarQube, 2023a]	272 [ESLint, 2023b]
Rule error levels	1. Minor 2. Major 3. Critical 4. Blocker	1. off 2. warn 3. error
Expendable (via plugins, custom rules, etc.)	• Custom Rules • Importable Rules	• Plugins • Custom Parsers • Formatters
Test Coverage	Yes	Yes (Via plugin [ESLint, 2023a])
CI integration	Yes	Yes (Via plugin [ESLint, 2023a])
PR Analysis	Yes	Yes (Via plugin [ESLint, 2023a])
Branch Analysis	Yes	Yes (Via plugin [ESLint, 2023a])

Table 1: *Comparison of ESLint and SonarQube.*

we will need to write tests for each edge case reported, in every conditional branches, the method takes, thus increasing testing complexity as a whole.

While *Responsibility Smells* and *Conditional Logic Smells* are tied to high level of combinatorial operation in a code or a complex set of conditions in one function, *Measured Smells* appear when a function, method or class is too long and contains too much responsibility by its sheer size. It leads to more complex test and thus decreases testability as a whole in the software artifact. In an interview with Icypeas Chief Technical Officer, Corentin Ribeyre, he reported that extracting responsibilities to follow the SRP should be the utmost priority when one needs to test a software artifact.

These smells are usually refactored by using the Extract Method [Fowler, 2018] refactoring technique, which is the most common technique used in pull request to further increase testability of a software artifact [Reich and Maalej, 2023]. As such, they need to be covered by the linter as much as possible. Some smells falling in that category are hard to check statically as they require some degree of abstraction not easily grasped by a static code checker (e.g. Fate Over Action) but these caveats can be avoided by adding more config-

uration options to their rules, facilitating their detection.

On the other hand, *Name Smells* violate readability and clarity in the code often obscuring the intent and/or the responsibility of a code block [Jerzyk and Madeyski, 2023]. In the case of white box testing², it is crucial to understand at a glance what a function, method or class is supposed to do. In each interview, clarity was the second reported factor hindering testability in a software artifact.

Related to data in a software, *Data Smells* reduce testability by either hiding class dependencies or mutating data at runtime requiring a more extensive test suit to tackle every edge cases. This increases complexity as a whole in the software artifact and will thus increase time spent on testing, often leading to Test-Refactoring Pair Pull Requests extracting values to allow mocking the passed data [Reich and Maalej, 2023].

Finally, nine of the ten interviewees reported that breaking the Don't Repeat Yourself (DRY) principle is, in fact, the third most common occurrence of low testability in a software

²Testing a method, a function or a class by knowing its exact intern logic.

artifact. It is a well known problem in software development and should be tackled by refactoring [Fowler, 2018]. As such, Duplicated Code is a smell to check for as well, to further increase testability in the artifact.

5 Building a Testability Linter

The idea of a linter is tied to the fact that developers in 99% of cases follow their linter’s rules faithfully throughout the development process of a software [Latapy *et al.*, 2023]. As such, we decided to implement a linter in Rust [Klabnik and Nichols, 2022] for its performances and memory safety while still retaining strong parallelization capabilities and a wide set of external libraries.

We decided to devise a simple prototype as a proof of concept first to test if it was indeed possible to statically enforce the rules I gathered throughout the internship.

5.1 First Prototype

To reduce the complexity of the scope for the first prototype, we decided to simply make it work on Java class files. It was implemented using the `classfile-parser` crate³: a parser for the Java Class Files implementation [Oracle, 2018].

The inner work of the linter uses `classfile-parser`, returning a constant pool as per the Class Files implementation [Oracle, 2018]. It then manipulates it to check for rules marked as true in the TOML config file. For some rules, it extracts some configuration parameters to apply in the rule validation.

When a rule passes, it is shown in green in the report sent to the standard output. Otherwise it is shown in red with a trace indicating which rule failed, and why. Figure 2 shows an example of report implemented in the prototype. Figure 1 shows the workflow of the first prototype.

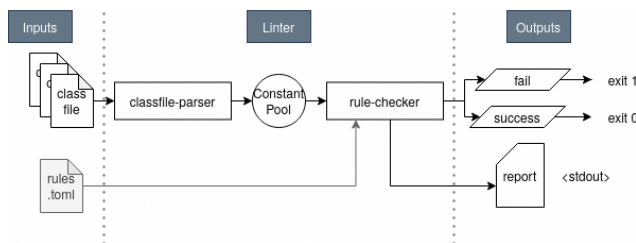


Figure 1: Workflow of the first Prototype.

For the prototype, we implemented this subset of 3 rules selectable in the configuration file:

- **No Binary Operators in Names:** a method using binary operator in its name, for example `doThisAndThat()` or `do_this_and_that` should be non-compliant to that rule and flagged as having too many responsibilities from its name, thus requiring extraction [Jerzyk and Madeyski, 2023].

³A crate is the smallest amount of code that the Rust compiler considers at a time [Klabnik and Nichols, 2022]. Usually we call a library “a crate” in the Rust ecosystem.

- **Max Number of Parameters:** a method exceeding the max number of parameters defined by this rule will fail, requiring to extract the parameters in a subtype [Jerzyk and Madeyski, 2023].
- **No Void Method Allowed:** a method with a return type of void will not comply with this rule and will fail [du Bousquet, 2010].

You can find the entire source code of the first prototype, tagged as version v0.2.0, on my GitLab repository⁴.

5.2 To a concrete evolution of the linter

Parsing Java Class Files will not be enough in the long run and poses some limitations:

- The user needs to actively compile Java files into Class Files;
- We are limited to only Java code bases;
- We can not refactor for multiple rules on different languages if we keep using the `classfile-parser` crate.

Hence, a perspective is to use the `rust-code-analysis` [Ardito *et al.*, 2020] crate for further evolutions of the linter. This crate features will allow us to further refine the linter. It is important to note that:

- `rust-code-analysis` is based on Tree-Sitter and thus is able to parse and build a concrete syntax tree for a source file even if it does not compile or run.
- This crate provides multiple languages parsing such as C, C++, Java, C#, Rust, JavaScript, Typescript, etc.
- It features inbuilt metrics like cyclomatic complexity or computing the number of lines of code, opening up the possibilities to a metric analysis as well as a pattern analysis.
- It features parallelization capabilities built in the crate directly.

As such, it is a scalable solution for the next evolutions of the linter.

Figure 3 shows the principle of the linter. It takes two inputs:

- A list of files containing the code source of the software artifact;
- A TOML file containing the rules and their configuration.

Each file is then parsed into a Concrete Syntax Tree (CST) using `rust-code-analysis` [Ardito *et al.*, 2020]. With the help of this CST, the linter verifies each rule following its configurations in the TOML file. It then outputs, on the standard output, a report explaining each violation found for each file.

A violation is tied to a level of gravity defined by the rule. The exit code of the linter depends on the highest gravity reported. We define the levels of gravity thereafter:

⁴<https://gitlab.com/Zhaith-Izaliel/testability-linter>

```
[FAIL] (file: tests/inputs/java/check_no_void/invalid/Example.class), Rules: No Void Method Allowed, (method: test3) - error: Rule Check
Failed, trace: This method has return type of void
[FAIL] (file: tests/inputs/java/check_no_void/invalid/Example.class), Rules: No Void Method Allowed, (method: test) - error: Rule Check
Failed, trace: This method has return type of void
[FAIL] (file: tests/inputs/java/check_no_void/invalid/Example.class), Rules: No Void Method Allowed, (method: test2) - error: Rule Check
Failed, trace: This method has return type of void
[OK] (file: tests/inputs/java/check_no_void/valid/Test.class), Rules: No Void Method Allowed
```

Figure 2: An example of report from the first prototype.

1. **info**: a failing rule with this gravity will appear as neutral information in the report, not requiring any action, terminating the linter with an exit code 0 if no higher gravity was reported;
2. **warning**: a failing rule will with this gravity show a warning in the report and terminate the linter with an exit code 2 if no higher gravity was reported;
3. **error**: A failing rule with this gravity will show an error on the report and the linter will terminate with an exit code 1.

With this implementation, the linter is easily interfaceable with multiple development processes.

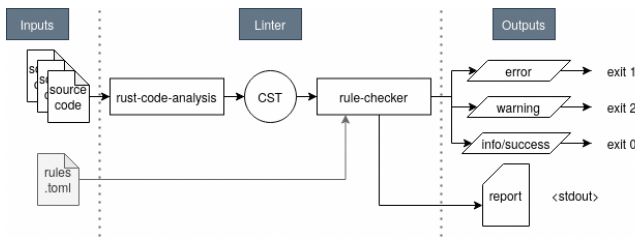


Figure 3: Workflow of the Testability Linter.

6 Results

Most testability related problems come from smelly code introducing unnecessary complexity and unpredictable behaviors, forcing a tester to usually guess the logic behind a function to be able to test it. Fortunately it is possible to describe code smells and to connect them to testability issues. The smells selected fall either into giving too many responsibilities to a single function, method or class, or obfuscate the code comprehension. In fact, these are the two most reported problems during the interviews alongside repeated code in a software artifact. Fortunately, these problems are easily verifiable by using a linter and are to some extent already checked in ESLint and SonarQube. It gives us a positive response to **RQ₁** and shows that we can statically check for 27 out of the 58 code smells gathered.

However, some code smells can not be detected this way as they provide some degree of abstraction not easily graspable by a linter. For example the *Dubious Abstraction* smell, happening when the levels of abstractions in a class are not constant and often dubious, is not testable statically. It requires manual intervention to fully understand to what extent the abstraction layers used in a class are dubious. Thus, requiring

ad hoc and often subjective insights [Jerzyk and Madeyski, 2023].

Thanks to our interviews and the literature review, we found out that code smells are the patterns (or in that case antipatterns) we should statically check in the linter. We were able to identify as many as 49 of them hindering testability. We then refined this list to 27 smells implementable as rules in the linter. The easiest code smells to check either violates SRP, DRY or code comprehension. Each of these characteristics can be statically enforced, even with our prototype. We can then answer positively to **RQ₂**.

7 Conclusions and Perspectives

Testability is at the core of software engineering. As such, offering the possibility to automatically, and systematically check for code smells hindering testability in a software artifact is a crucial step in the development life cycle of an application. By automating that process we are able to reduce the cost of tests development and maintenance without increasing time spent on testing considerations.

We developed a prototype linter in Rust [Klabnik and Nichols, 2022] implementing 3 rules to check for, in a code base, on Java class files [Oracle, 2018]. You can find the entire source code of the first prototype, tagged as version v0.2.0, on my GitLab repository⁵.

As of now, the linter is not suitable to be deployed as a full scale linter in the industry, but it is a fair base to build upon and create, in the end, a professional grade linter focused solely on testability to, hopefully, make software testing more accessible to new developers while giving them healthier coding habits.

As a perspective, we could develop a second linter implementing all the 27 rules we defined. We could also extend the list of rules with other code smells and also other antipatterns hindering testability as a whole. These could be gathered through more interviews with a different approach than we took. After a suitable amount of rules has been integrated, we could evaluate the linter in the field by spreading its use to professional software engineers and ask for their feedbacks.

⁵<https://gitlab.com/Zhaith-Izaliel/testability-linter>

References

- [Ardito *et al.*, 2020] Luca Ardito, Luca Barbato, Marco Castelluccio, Riccardo Coppola, Calixte Denizet, Sylvestre Ledru, and Michele Valsesia. rust-code-analysis: A rust library to analyze and extract maintainability information from source codes. *SoftwareX*, 12:100635, 2020.
- [Binder, 1984] Robert V. Binder. Design for testability in object-oriented systems. *Communication of the ACM*, 37(9):1, September 1984.
- [cplusplus.com, 2023] cplusplus.com. Tutorials: Template, 2023.
- [du Bousquet, 2010] Lydie du Bousquet. A new approach for software testability. In Leonardo Bottaci and Gordon Fraser, editors, *Testing – Practice and Research Techniques*, pages 207–210, Berlin, Heidelberg, 2010. Springer Berlin Heidelberg.
- [ESLint, 2023a] ESLint. Eslint documentation, 2023.
- [ESLint, 2023b] ESLint. Rules reference, 2023.
- [Fowler, 2018] Martin Fowler. *Refactoring*. Addison-Wesley Signature Series (Fowler). Addison-Wesley, Boston, MA, 2 edition, 2018.
- [Jerzyk and Madeyski, 2023] Marcel Jerzyk and Lech Madeyski. *Code Smells: A Comprehensive Online Catalog and Taxonomy*, pages 543–576. Springer Nature Switzerland, Cham, 2023.
- [Klabnik and Nichols, 2022] Steve Klabnik and Carol Nichols. *The Rust Programming Language, 2nd Edition*. No Starch Press, San Francisco, California, 2022.
- [Latappy *et al.*, 2023] Corentin Latappy, Quentin Perez, Thomas Degueule, Jean-Rémy Falleri, Christelle Urtado, Sylvain Vauttier, Xavier Blanc, and Cédric Teyton. Mlinter: Learning coding practices from examples-dream or reality?, 2023.
- [Myers *et al.*, 2011] Glenford J. Myers, Corey Sandler, and Tom Badgett. *The Art Of Software Testing, 3rd Edition*. John Wiley & Sons, Hoboken, New Jersey, 2011.
- [Oracle, 2018] Oracle. Chapter 4. the class file format, 2018.
- [Reich and Maalej, 2023] Pavel Reich and Walid Maalej. Testability refactoring in pull requests: Patterns and trends, 2023.
- [Shaheen, 2009] Muhammad Rabee Shaheen. Validation de métriques de testabilité logicielle pour les programmes objets. 10 2009.
- [SonarQube, 2023a] SonarQube. Sonar rules, 2023.
- [SonarQube, 2023b] SonarQube. Sonarqube documentation, 2023.
- [SonarQube, 2023c] SonarQube. Sonarqube website, 2023.
- [Tómasdóttir *et al.*, 2020] Kristín Fjóra Tómasdóttir, Maurício Aniche, and Arie Van Deursen. The adoption of javascript linters in practice: A case study on eslint. *IEEE Transactions on Software Engineering*, 46(8):863–891, 2020.
- [Wake, 2004] W.C. Wake. *Refactoring Workbook*. Addison-Wesley, 2004.

A Appendix

A.1 Linter rules

Responsibility Smells

- **Feature Envy**: "If a method inside a class manipulates more features (be it fields or methods) of another class more than from its own, then this method has a Feature Envy" [Jerzyk and Madeyski, 2023].
- **Parallel Inheritance**: "This occurs when an inheritance tree depends on another inheritance tree by composition, and to create a subclass for a class, one finds that he has to make a subclass for another class" [Jerzyk and Madeyski, 2023].
- **Combinatorial Explosion**: "This occurs when a lot of code does almost the same thing" [Jerzyk and Madeyski, 2023].
- **Conditional Complexity**: "In data-oriented programming, the usage of switch-like statements (lengthy, cascading `if` statements or `switch/case`) should be relatively rare. One such `switch` usually executes code scattered around the code base and should usually be replaced with a polymorphism solution" [Jerzyk and Madeyski, 2023].
- **Afraid To Fail**^{*}: "If it is expected that a method might fail, then it should fail, either by throwing an Exception or, if not - it should return a special case `None/Null` type object of the desired class (following the *Null Object Pattern*), not null itself" [Jerzyk and Madeyski, 2023].
- **Side Effects**^{*}: No function or method should do more than what they tell [Jerzyk and Madeyski, 2023].
- **Fate Over Action**^{*}: "In Object-Oriented Programming, classes and their data go hand in hand with behavior. If a class has only the first part, it is an indicator that there could be a functionality tied to these variables, but it is missing or placed somewhere else" [Jerzyk and Madeyski, 2023].

Name Smells

- **Binary Operator in Names**: Methods or functions names having `and` or `or` in their names shows they do not comply to the SRP and as such, should be changed [Jerzyk and Madeyski, 2023].
- **Boolean Blindness**: Using a boolean directly as `true/false` hides its meaning and as such should be replaced by a type explaining what both states mean [Jerzyk and Madeyski, 2023].
- **Complicated Regexp Expression**: Regular Expressions are usually complex to understand. When possible, these long regexes should be divided in multiple smaller regex connected via string interpolation with helpful names [Jerzyk and Madeyski, 2023].
- **Fallacious Method Name**^{*}: "This smell is caused by creating conflicting methods or functions regarding their functionality and naming." [Jerzyk and Madeyski, 2023]. For example, naming a method `getFoods()` $\rightarrow$ `Food` returning a single value is confusing since its name is plural. One should expect a list

to be returned instead: `getFoods()` $\rightarrow$ `list[Food]` [Jerzyk and Madeyski, 2023].

- **Magic Numbers**: Numbers used, for example, as a bound of an iterative loop, should be stored in a variable or constant clearly named to express its intent.

Measured Smells

- **Large Class**^{*}: A large class is usually feature heavy and contains too many responsibilities to test for and should be avoided [Jerzyk and Madeyski, 2023].
- **Long Method**^{*}: A long method or function breaks the SRP in most cases and should be extracted in multiple sub-functions [Jerzyk and Madeyski, 2023].
- **Long Parameters List**^{*}: A long parameter list in a method or a function implies a larger scope than its responsibility. We should refrain from making too many parameters and extract them in types instead [Jerzyk and Madeyski, 2023].

Object-Oriented Abusers

- **Refused Bequest**[†]: "Whenever a subclass inherits from a parent but only uses a subset of the implemented parent methods, that is called Refused Bequest" [Jerzyk and Madeyski, 2023].
- **Base Class depends on Subclass**: "The rule is that the child classes should be deployable independently of the parent class. That allows us to deploy the system in discrete and independent components. When one of these subclasses is modified, the base class does not need to be redeployed as well" [Jerzyk and Madeyski, 2023].

Conditional Logic Smells

- **Flag Argument**: "A kind of function argument that tells the function to carry out a different operation depending on its value" [Fowler, 2018].
- **Special Cases**^{*}: "A complex `if` statement and/or value checking before doing the actual work of a function" [Wake, 2004].
- **Callback Hell**: "The callback is a function that is passed into another function as an argument that is meant to be executed later. Alone in separation, they are not causing or indicating any problems. Instead, the long list of chained callbacks is something to watch out for" [Jerzyk and Madeyski, 2023].
- **Null Check**^{*}: "Null check is widespread everywhere because the programming languages allow it. It causes a multitude of `undefined` or `null` checks everywhere: in guard checks, in condition blocks, and verifications clauses. Instead, special objects could be created that implement the missing-event behavior, errors could be thrown and caught, and many duplications would be removed" [Jerzyk and Madeyski, 2023].

[†]This rule should allow a degree of lenience as, in reality, not everything in a parent class is usable for the children (following a specialization principle).

Data Smells

- **Data Clump:** "Data Clumps refer to a situation in which a few variables are passed around many times in the codebase instead of being packed into a separate object" [Jerzyk and Madeyski, 2023].
- **Hidden Dependencies:** "Hidden Dependency is a situation where, inside a class, methods are silently resolving dependencies, hiding that behavior from the caller" [Jerzyk and Madeyski, 2023].
- **Mutable Data*:** "Mutable data are harmful because they can unexpectedly fail other parts of the code. It is a rich source of bugs that are hard to spot because they can occur under rare conditions" [Jerzyk and Madeyski, 2023].

Other Rules

- **Duplicated Code:** Duplicated code violates the Don't Repeat Yourself (DRY) principle and should be avoided at all costs [Jerzyk and Madeyski, 2023].
- **Always Have Setters and Getters:** Setters and Getters are important when testing a class in object-oriented programming and enforce automatic checks when accessing or modifying attributes. They should always be present, at least for test purposes [du Bousquet, 2010].
- **No Void Method/Function:** void methods and functions are part of a programming language as a whole but shadow useful information, for instance, if the method accessing a hash map found an item using the provided key. As such, for test purposes, this rule disallows the use of void as a return type in methods and functions [du Bousquet, 2010].

A.2 Interview Template

Interviewé

- Prénom Nom
- Date: DD/MM/AAAA
- Durée: HH:MM
- Enregistrement (si applicable)
- Expérience dans le développement:
- Expérience dans le test:
- Nom de l'entreprise:
- Type d'entreprise:
- Taille:

Check-list préliminaire

- ☐ Explication de l'étude
- ☐ Info sur les données traitées (engagement envoyer rapport de stage si nécessaire)
- ☐ Consentement enregistrement
- ☐ Consentement citation société

*These rules contain caveats and should be implemented using specific configuration options to allow further granularity when used to check code statically.

Description de l'étude

- Étude concernant la testabilité des logiciels et des pratiques de développement permettant de rendre une application testable et robuste
- Création d'un linter sur plusieurs langages (Java/C#/JS/Rust/etc.) pour vérifier qu'un code suit des pratiques données dans des règles définies pour qu'il soit testable

Questions

1. Comment intégrez-vous vos stratégies de test dans vos processus de développement ?
2. Dans quels langages et/ou avec quels frameworks travaillez-vous ?
3. Quels outils utilisez-vous pour tester vos applications ?
4. Selon l'International Software Testing Qualifications Board, 4 niveaux de tests sont effectués dans des applications:
 - **Unit Testing:** *tester une fonctionnalité spécifique à une section du code;*
 - **Integration Testing:** *tester les interfaces de composants, librairies, API, etc;*
 - **System Testing:** *tester un système complètement intégré pour vérifier qu'il répond à ses exigences;*
 - **Acceptance Testing:** *tester le logiciel pour s'assurer qu'il répond aux attentes fonctionnelles.*

Quels sont les niveaux de tests que vous effectuez pour vos applications dans votre entreprise ?

5. Et personnellement ?
6. Est-ce que vous avez des stratégies pour choisir vos tests (techniques, méthodes, métriques, etc.) ?
7. Votre entreprise met-elle en œuvre des méthodes ou a-t-elle des outils pour évaluer la qualité des tests et/ou leur bonne exécution ? *Selon la réponse: Est-ce automatisé/systématique ?*
8. En tant que testeur, avez-vous remarqué des pratiques (constructions, structures dans le code, etc.) qui facilitent votre travail de testeur ou, qui, au contraire, rendent votre travail plus complexe ?
9. Seriez-vous intéressé par un outil d'analyse de code statique (un linter à la manière de SonarQube ou ESLint par exemple) pour valider la mise en place de ces pratiques (constructions, structures dans le code, etc.) de façon automatique ? Si oui, quels sont les pratiques que vous voudriez voir couvertes, ainsi que les langages concernés ?